BASE DE DATOS II

Unidad 8: Base de Datos para Dispositivos Móviles e Integración

Informe — Trabajo Práctico 10

Food Store — Sincronización SQLite ↔ PostgreSQL para tabletas

Institución	Facultad — Base de Datos II
Curso	Base de Datos II
Unidad	Unidad 8 — Base de Datos para Dispositivos Móviles e Integración
Trabajo	Trabajo Práctico 10 — Food Store (SQLite)
Integrantes	Ramírez, Deseff, Vargas, Godoy
Repositorio	https://github.com/FacundoDeseff/Tps_BasedeDatos

Este informe documenta el diseño, la implementación y la verificación del trabajo práctico. Se redactó en tercera persona, en estilo objetivo, y se conserva todo el contenido técnico sin omisión de casos ni consignas.

Tabla de Contenidos

Placeholder for table of contents 0

1. Introducción y Alcance

El presente informe documenta el Trabajo Práctico 10 de la asignatura Base de Datos II, correspondiente a la Unidad 8: Base de Datos para Dispositivos Móviles e Integración. El trabajo aborda el desarrollo de una aplicación para tablet que opera en una feria con conectividad intermitente, manteniendo una copia parcial del catálogo de productos en una base de datos SQLite local, gestionando borradores de pedido sin conexión y sincronizándose con el servidor PostgreSQL al restablecerse la señal.

La arquitectura propuesta sigue el principio de que la tableta propone y el servidor dispone: el dispositivo móvil nunca modifica precios, stock ni confirma ventas; esas decisiones residen exclusivamente en el servidor. El informe describe el estado de las cuatro consignas (C1-C4), los datos que viajan de PostgreSQL a SQLite, el mecanismo de sincronización con resolución de conflictos mediante un semáforo, los casos de prueba C2 a C6, la verificación de invariantes en PostgreSQL, el mapa de la capa de datos, el plan de mantenimiento y el code review asistido por inteligencia artificial.

2. Estado del Trabajo por Consigna

El trabajo se organizó en cuatro consignas. Todas se encuentran completadas y verificadas. El cuadro siguiente resume el estado de cada una.

Consigna	Descripción	Estado	Evidencia
C1	Esquema SQLite STRICT con 4 tablas, pruebas de transacción, catálogo sin servidor y datos inválidos (C7).	Completada	movil/schema_sqlite.sql, movil/pruebas_sqlite.sql
C2	Sincronización: vista v_snapshot_movil, CSV, semáforo, casos C2-C6, verificación PostgreSQL antes/después.	Completada y probada	movil/simulador.py, docs/evidencia_consigna2.txt
C3	Mapa de capa de datos, plan de mantenimiento, code review con IA (F1-F12).	Completada	docs/verificaciones_consigna3.sql, docs/evidencia_consigna3.txt
C4	Spec en Kiro, bitácora de IA, DUIA, README y ensayo de defensa.	Completada	movil/spec-kiro.md, docs/BITACORA.md, docs/DUIA.md

Durante la revisión de la consigna 3 se corrigieron dos casos borde: un borrador sin ítems que el semáforo aprobaba erróneamente (hallazgo F4) y productos viejos que permanecían como disponibles tras una instantánea más angosta (hallazgo F5). Ambos fueron corregidos en el código y verificados contra la base de datos.

2.a Qué Datos Viajan a la Tablet

Durante la operación de la feria, la tablet requiere una copia parcial del sistema. Se define qué información viaja desde la base principal en PostgreSQL hacia la tabla local producto_cache en SQLite, y qué datos quedan excluidos, con su respectiva justificación.

Tabla de correspondencia de columnas

Columna en PostgreSQL	¿Viaja?	En la tablet	Justificación
id_producto	Sí	id	Identifica el producto en borradores y sincronización.
nombre	Sí	nombre	Permite mostrar el catálogo en pantalla.
precio_lista (NUMERIC 10,2)	Sí	precio_centavos (INTEGER)	Se convierte a centavos enteros para evitar errores de redondeo con decimales.
stock	Sí	stock_visto	Solo orientativo: es el stock observado al momento de la copia.
eliminado	Sí	eliminado	Lápida de baja lógica: la tablet debe conocer que un producto se dio de baja.
(derivada) stock > 0 AND NOT eliminado	Sí	disponible	Permite filtrar el catálogo sin recalcular.
(generada en la tablet)	No	fecha_copia	Indica la antigüedad de la copia.
id_categoria	No	—	No es necesaria para vender en la feria; puede agregarse si se muestra por categoría.
Tabla usuario (nombre, apellido)	No	—	Datos personales: no deben residir en un dispositivo que puede extraviarse. La tablet solo guarda un alias en pedido_borrador.usuario.
Tablas pedido y detalle_pedido	No	—	Son ventas confirmadas decididas por el servidor. La tablet solo gestiona borradores.

Se decidió enviar únicamente lo mínimo necesario para mostrar el catálogo y armar borradores sin conexión. Cuantos menos datos sensibles residan en la tablet, menor es el riesgo en caso de extravío del dispositivo. La fuente de verdad es PostgreSQL, ya que el precio, el stock y las ventas los decide el servidor; la copia de la tablet puede encontrarse desactualizada, por lo que lo que la tablet arma es una propuesta que el servidor valida al sincronizar.

Nota sobre el índice único parcial ux_cola_una_pendiente

Durante las pruebas de reenvío, se observó que SQLite prioriza la validación del índice frente a un envío que permanece en estado PENDIENTE, arrojando un error sobre id_local. Una vez que la solicitud se procesa y deja de estar pendiente, actúa la clave primaria de solicitud_uuid, garantizando la idempotencia del sistema.

3. Sincronizar y Resolver

3.a La Instantánea

Para no modificar la base original, se creó una copia en PostgreSQL denominada foodstore_u8, obtenida mediante CREATE DATABASE foodstore_u8 TEMPLATE foodstore;. Sobre dicha copia se definió la vista v_snapshot_movil, que agrupa la información mínima requerida para la feria: identificador, nombre, precio en centavos, stock, disponibilidad, lápida de baja lógica y fecha de la copia.

Vista v_snapshot_movil (PostgreSQL, archivo servidor_snapshot.sql)

```
CREATE VIEW v_snapshot_movil AS

SELECT

p.id_producto AS id,

p.nombre AS nombre,

round(p.precio_lista * 100)::bigint AS precio_centavos,

p.stock AS stock,

(p.stock > 0 AND NOT p.eliminado) AS disponible,

p.eliminado AS eliminado,

now() AS fecha_copia

FROM producto p

WHERE p.id_producto IN (1, 3, 4, 5, 6, 7, 8, 10);
```

La feria de este trabajo opera con ocho productos, por lo que la vista se limita a ese catálogo. La copia de la tablet es parcial, tal como lo exige la consigna. Las bajas lógicas no se filtran: deben viajar para que la tablet tenga conocimiento de que un producto ya no se comercializa.

Para transferir la instantánea a SQLite se exporta el CSV mediante \copy ... CSV HEADER y se importa con .import --csv --skip 1 servidor_snapshot.csv snapshot_servidor. PostgreSQL escribe los booleanos como 't'/'f', por lo que snapshot_servidor los guarda como texto y cargar_cache.sql los convierte a 0/1 al llenar producto_cache. Ese llenado utiliza UPSERT para no eliminar filas que aún tienen ítems de borradores referenciándolas. La copia base contiene 8 productos y su fecha de copia es 2026-11-04.

Tabla snapshot_servidor (staging del CSV, archivo importar_snapshot.sql)

```
CREATE TABLE snapshot_servidor (

id INTEGER PRIMARY KEY,

nombre TEXT NOT NULL,

precio_centavos INTEGER NOT NULL CHECK (precio_centavos >= 0),

stock INTEGER NOT NULL CHECK (stock >= 0),

disponible TEXT NOT NULL CHECK (disponible IN ('t', 'f')),

eliminado TEXT NOT NULL CHECK (eliminado IN ('t', 'f')),

fecha_copia TEXT NOT NULL

) STRICT;
```

Carga de producto_cache con UPSERT (archivo cargar_cache.sql)

```
INSERT INTO producto_cache (

id, nombre, precio_centavos, stock_visto, disponible, eliminado, fecha_copia

)

SELECT

id, nombre, precio_centavos, stock,

CASE disponible WHEN 't' THEN 1 ELSE 0 END,

CASE eliminado WHEN 't' THEN 1 ELSE 0 END,

fecha_copia

FROM snapshot_servidor

WHERE 1

ON CONFLICT (id) DO UPDATE SET

nombre = excluded.nombre,

precio_centavos = excluded.precio_centavos,

stock_visto = excluded.stock_visto,

disponible = excluded.disponible,

eliminado = excluded.eliminado,

fecha_copia = excluded.fecha_copia;


UPDATE producto_cache

SET disponible = 0

WHERE EXISTS (SELECT 1 FROM snapshot_servidor)

AND id NOT IN (SELECT id FROM snapshot_servidor);
```

3.b El Semáforo de Sincronización

El archivo semaforo.sql evalúa únicamente las solicitudes que permanecen en estado PENDIENTE y las compara con snapshot_servidor. Se aplican las siguientes reglas:

Estado	Color	Condición
RECHAZADO	Rojo	El borrador no tiene ítems, o algún producto no existe en la instantánea, está dado de baja (eliminado = 't'), no está disponible o no alcanza el stock.
CONFLICTO	Amarillo	No hay rechazo, pero cambió al menos un precio respecto a la instantánea.
SIMULADO_APROBADO	Verde	Todos los ítems coinciden con la instantánea.

La consulta realiza un LEFT JOIN de item_borrador con snapshot_servidor y emplea un CASE con conteos por solicitud. La cláusula WHERE estado = 'PENDIENTE' es fundamental: al repetir la simulación no se modifica nada, lo que brinda idempotencia en la simulación. El script no toca PostgreSQL; solo escribe en cola_sync de SQLite. SIMULADO_APROBADO no es una venta: es una propuesta que posteriormente debe validar el servidor.

Semáforo semaforo.sql (SQLite)

```
UPDATE cola_sync

SET estado = CASE

WHEN NOT EXISTS (

SELECT 1 FROM item_borrador i WHERE i.id_local = cola_sync.id_local

) THEN 'RECHAZADO'


WHEN (

SELECT COUNT(*)

FROM item_borrador i

LEFT JOIN snapshot_servidor s ON s.id = i.producto_id

WHERE i.id_local = cola_sync.id_local

AND (

s.id IS NULL

OR s.eliminado = 't'

OR s.disponible = 'f'

OR i.cantidad > s.stock

)

) > 0 THEN 'RECHAZADO'


WHEN (

SELECT COUNT(*)

FROM item_borrador i

JOIN snapshot_servidor s ON s.id = i.producto_id

WHERE i.id_local = cola_sync.id_local

AND i.precio_centavos_visto <> s.precio_centavos

) > 0 THEN 'CONFLICTO'


ELSE 'SIMULADO_APROBADO'

END,

motivo = CASE

WHEN NOT EXISTS (

SELECT 1 FROM item_borrador i WHERE i.id_local = cola_sync.id_local

) THEN 'borrador sin ítems: no hay nada para sincronizar'


WHEN (...) > 0 THEN (SELECT group_concat(detalle, '; ') FROM (...))
```

```
WHEN (...) > 0 THEN (SELECT group_concat('producto ' || i.producto_id || ': ' ||
i.precio_centavos_visto || ' -> ' || s.precio_centavos, '; ')
FROM ...)

ELSE 'todo coincide con la instantánea'
END
WHERE estado = 'PENDIENTE';
```

3.c Los Casos de Prueba (C2 a C6)

Se ejecutaron cinco casos con python movil/simulador.py. El simulador arma la base SQLite, importa la instantánea de cada caso y ejecuta semaforo.sql. Todos los casos siguen el mismo patrón de creación de borrador y solicitud, variando solo los valores:

Patrón de creación de borrador y solicitud (SQL)

```
INSERT INTO pedido_borrador (id_local, usuario, forma_pago)
VALUES ('', 'cliente_demo', 'EFECTIVO');

INSERT INTO item_borrador (id_local, producto_id, cantidad, precio_centavos_visto)
VALUES ('', , , );

INSERT INTO cola_sync (solicitud_uuid, id_local)
VALUES ('', '';
```

Caso C2 — Precio cambiado

Se crea un borrador del producto 1 con 2 unidades a 125.945 centavos. Posteriormente se incrementa el precio en el servidor (UPDATE producto SET precio_lista = precio_lista + 100.00 WHERE id_producto = 1;), se exporta la instantánea nueva y se ejecuta el semáforo. El resultado es CONFLICTO | producto 1: 125945 -> 135945. El cliente acepta el nuevo precio, se actualiza item_borrador.precio_centavos_visto y se crea una solicitud nueva, que obtiene SIMULADO_APROBADO. La solicitud anterior permanece en CONFLICTO para revisión.

Caso C3 — Más unidades que el stock disponible

Se arma un borrador de 192 unidades del producto 3, mientras el servidor dispone de 142. El resultado es RECHAZADO | producto 3: stock 142 < 192.

Caso C4 — Producto dado de baja

Se crea un borrador del producto 4 y posteriormente se aplica una baja lógica en el servidor (UPDATE producto SET eliminado = true WHERE id_producto = 4;). El resultado es RECHAZADO | producto 4 dado de baja. Al recargar el catálogo, el producto 4 ya no aparece entre los disponibles, pero la fila permanece en producto_cache con eliminado = 1 (lápida) y disponible = 0.

Caso C5 — Sin cambios

Se crea un borrador del producto 5 sin modificar el servidor. El resultado es SIMULADO_APROBADO | todo coincide con la instantánea. Al ejecutar el semáforo dos veces consecutivas, el resultado es idéntico, porque la consulta solo evalúa solicitudes en estado PENDIENTE.

Caso C6 — Reenvío del mismo UUID

Se intenta insertar la misma solicitud con el mismo solicitud_uuid. Se obtiene el error UNIQUE constraint failed: cola_sync.solicitud_uuid; la cola mantiene una sola solicitud y repetir la simulación no reevalua nada. En un sistema productivo, la protección no puede residir solo en la tablet: el servidor debe recibir solicitud_uuid como clave de idempotencia, registrarlo en una tabla de solicitudes procesadas dentro de la misma transacción que crea el pedido, y si el UUID ya existe, responder OK sin volver a vender.

Verificación en PostgreSQL (antes y después)

Antes y después de la simulación se consultaron los pedidos y el stock total:

Consulta de invariantes (PostgreSQL)

```
SELECT (SELECT count(*) FROM pedido) AS pedidos,

(SELECT coalesce(sum(stock), 0) FROM producto) AS stock_total;
```

Momento	Pedidos	Stock total
Antes de la simulación	200.000	4.998.378
Después de la simulación	200.000	4.998.378

La simulación no modificó ni los pedidos ni el stock total, ni en la copia foodstore_u8 ni en la original foodstore. La tablet nunca cambió el stock ni confirmó ventas. El cambio de precio de C2 y la baja de C4 se realizaron en la copia del servidor para preparar los casos, no por el semáforo.

4. Integrar y Revisar

4.a Mapa de la Capa de Datos

Cada motor cumple un papel distinto y tiene un dueño distinto para cada dato. La regla de oro se mantiene: el móvil propone, el servidor dispone.

Dato	Motor que manda (fuente de verdad)	Copia o derivado	Quién escribe
Catálogo: nombre, precio_lista, stock, eliminado	PostgreSQL (producto, categoria)	producto_cache en SQLite (instantánea parcial); items[].nombre_producto y items[].categoria en MongoDB (snapshot histórico)	Solo PostgreSQL
Clientes	PostgreSQL (usuario)	Snapshot cliente embebido en cada pedido de MongoDB; en SQLite no viaja (solo el alias pedido_borrador.usuario)	Solo PostgreSQL
Ventas confirmadas	PostgreSQL (pedido + detalle_pedido)	Colección foodstore_nosql.pedidos en MongoDB (agregado de lectura)	Solo el servidor
Borrador en curso	SQLite (pedido_borrador, item_borrador)	—	La tablet; es una propuesta
Estado del envío	SQLite (cola_sync) en la simulación; en producción, tabla de solicitudes procesadas en PostgreSQL	—	La tablet encola; el servidor resuelve
Precio histórico de una línea	PostgreSQL (detalle_pedido.subtotal)	MongoDB lo deriva y lo congela en items[].precio_unitario_historico y items[].subtotal	El servidor al cerrar la venta

PostgreSQL es la fuente de verdad del negocio: catálogo, precio, stock y ventas. La tablet nunca cambia esos datos; el semáforo solo escribe en cola_sync. MongoDB (Unidad 7) guarda el pedido como agregado documental (cabecera + cliente + items[]). Es una copia de lectura, derivada de PostgreSQL, que no decide precio, stock ni ventas y puede regenerarse. Su valor es el snapshot histórico: conserva el nombre, la categoría y el precio implícito del momento de la compra. SQLite (tablet) guarda lo mínimo para vender sin señal: catálogo parcial, borradores y cola de envío.

4.b Plan de Mantenimiento

Se respalda foodstore completo, una vez por día a las 03:00, con pg_dump -Fc y escritura en .parcial + renombre. El respaldo incluye la tabla de solicitudes procesadas que garantiza la idempotencia; sin ella, un reenvío posterior a una restauración podría duplicar una venta. Las tablets no se respaldan: producto_cache se reconstruye desde una instantánea y los borradores son descartables porque el servidor no confirmó nada. MongoDB tampoco es crítico: se puede regenerar desde PostgreSQL; si en producción pasara a ser dueño de algún dato, habría que agregar mongodump y su prueba de restauración.

La restauración se prueba todas las semanas en foodstore_restore_test (nunca sobre foodstore) con restore_test.ps1: conteos contra el respaldo, CHECKs, claves foráneas huérfanas, pedido.total = SUM(detalle_pedido.subtotal) y secuencias al día, más una prueba negativa que debe fallar. Se registran RTO y RPO. Para cerrar la capa móvil, después de restaurar se vuelve a crear v_snapshot_movil y se repiten C3 y C5 en SQLite: el rojo y el verde deben dar lo mismo que antes.

Si se restaura un respaldo viejo de PostgreSQL, el servidor queda atrás y la tablet puede tener datos más nuevos. Al reconectar, el servidor no confía en SIMULADO_APROBADO ni en los precios vistos: descarta la instantánea local, exige una copia completa y vuelve a evaluar cada solicitud pendiente contra el estado restaurado. Si el solicitud_uuid ya está en la tabla de procesadas del respaldo, responde OK sin volver a vender; si no está, la solicitud pudo haberse cobrado en la ventana perdida y va a revisión manual, no se aplica automáticamente.

4.c Code Review con Inteligencia Artificial

Se solicitó a OpenCode (modo plan, solo lectura) que revisara todos los scripts de la cursada (tp1 a tp10), con la condición de que cada hallazgo fuera comprobable en la base. Cada uno se verificó ejecutando SQL de solo lectura sobre foodstore o sobre una base SQLite temporal; las consultas están en docs/verificaciones_consigna3.sql y la salida completa en docs/evidencia_consigna3.txt. Se descartaron dos falsos positivos. Las decisiones se clasifican como: aceptado (hallazgo válido documentado como deuda), corregido (se modificó el código) y descartado (falso positivo).

#	Hallazgo (archivo)	Decisión	Motivo
F1 — TP3	carga_masiva.sql:25-27: el LATERAL no referencia la fila y los 50.000 productos quedaron en una sola categoría	Aceptado	Defecto real de los datos sintéticos; regenerar la carga invalidaría mediciones y evidencias de TP3/TP5/TP6/TP8. Se documenta como limitación.
F2 — TP6	tp_desnormalizacion_top_categorias.sql:22,29,44: la vista 'diaria' agrupa por timestamp y fecha = CURRENT_DATE solo matchea la medianoche	Aceptado	La definición correcta es ped.fecha::date. No se recrea la vista para no alterar la evidencia entregada del TP6.
F3 — TP4	queries_tp4.sql:22,36: la facturación histórica filtra pr.eliminado = FALSE y subestima las ventas	Aceptado	El reporte es histórico y TP5/TP6 no aplican ese filtro. Se documenta la inconsistencia.

#	Hallazgo (archivo)	Decisión	Motivo
F7 — TP3	carga_masiva.sql:44: el subtotal sintético no se calcula con precio_lista	Aceptado	Es una característica de los datos, no un error del modelo.
F4 — TP10	semaforo.sql:27-31: un borrador sin ítems daba SIMULADO_APROBADO	Corregido	Se agregó la rama NOT EXISTS (item_borrador): ahora da RECHAZADO con motivo 'borrador sin ítems'.
F5 — TP10	cargar_cache.sql:44-47: una instantánea más angosta dejaba productos viejos como disponibles	Corregido	Se agrega UPDATE ... SET disponible = 0 para los ids que ya no vienen. No se toca eliminado.
F11 — TP10	importar_snapshot.sql:23-24: los booleanos del CSV se guardan como TEXT	Descartado	Es la tabla de staging del CSV (t/f); cargar_cache.sql los pasa a 0/1 y producto_cache tiene CHECK ... IN (0,1).
F12 — TP3	equivalencias.sql:5,16: COUNT(p.id_producto) parece un error frente a COUNT(*)	Descartado	Con LEFT JOIN, COUNT(column) es lo correcto: devuelve 0 en categorías vacías. Cambiarlo a COUNT(*) sería el bug.

F4 y F5 fueron corregidos en semaforo.sql y cargar_cache.sql, respectivamente, y verificados contra la base. F11 y F12 se descartaron como falsos positivos tras la verificación. El hallazgo más significativo fue F4: la versión inicial del semáforo aprobaba borradores sin ítems porque los dos conteos daban 0 y el CASE caía en SIMULADO_APROBADO. Se ordenó el CASE de lo más restrictivo a lo más permisivo, exigiendo explícitamente el caso vacío.

5. Conclusión

El trabajo práctico 10 se completó en su totalidad. Las cuatro consignas están verificadas contra la base de datos real. El esquema SQLite cumple con los requisitos STRICT, las claves foráneas activas y los CHECKs de integridad. La vista v_snapshot_movil define la copia parcial que viaja a la tablet, el semáforo resuelve los casos C2 a C6 con los resultados esperados, y los invariantes de PostgreSQL (pedidos y stock total) se mantienen inalterados antes y después de la simulación. El code review asistido por IA identificó y corrigió dos casos borde críticos (F4 y F5), documentando además los hallazgos de otros trabajos prácticos.

La arquitectura adoptada —tablet como proponente y servidor como decisor— garantiza que el dispositivo móvil pueda operar sin conexión constante sin comprometer la integridad ni la consistencia del negocio. La idempotencia por UUID inicia en la tablet y debe cerrarse en el servidor dentro de la transacción de venta.